CSE 554 Machine Learning Systems

Lecture 13 - Batching

Baris Kasikci

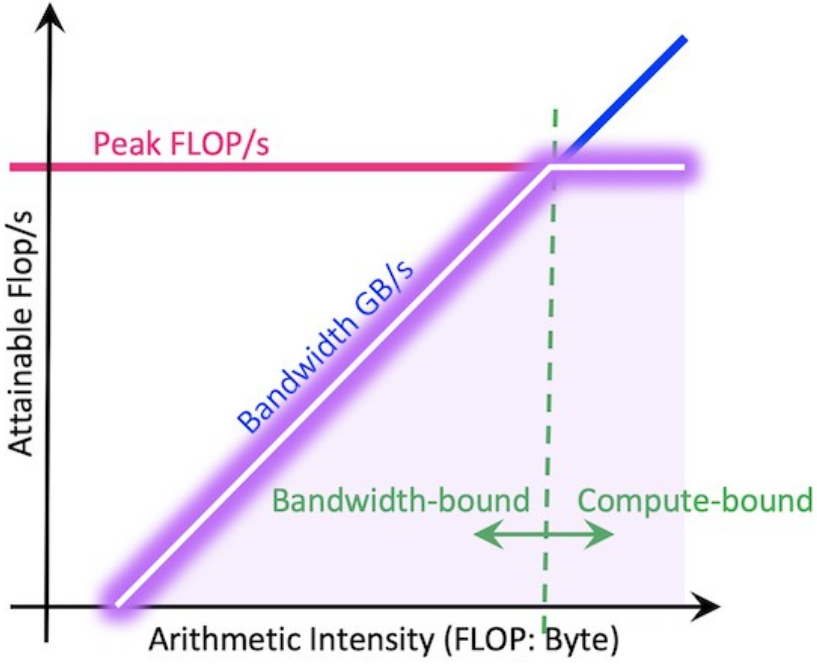
BE BOUNDLESS

Class Overview

1. GPU Architecture and Programming
2. Transformers (Architecture and Implementation)
3. Performance Modeling
4. Quantization
5. Memory Management
6. NVIDIA Blackwell Architecture
7. Sparsity
8. Mixture of Experts (MoE)
9. Speculative decoding
10. **Batching**
11. Parallelism
12. RL for LLMs
13. Agentic systems

Batching is important in LLM serving

H100 needs a batch size of 333 to reach peak performance



How to form a batch?

Consideration 1:
User Experience

Consideration 2:
Throughput

How to define user experience

- Time to First Token (TTFT)
  - The time between user submitting the request and the generation of first token
  - TTFT = Queuing time + Prefill time
- Time Per Output Token (TPOT/ITL/IBT)
  - The time between each output token
- Average TPOT
  - The average time of producing one output token
- End to end time
  - The time to queue, prefill and decode the whole request
- Normalized Latency
  - End-to-end time / number of output tokens

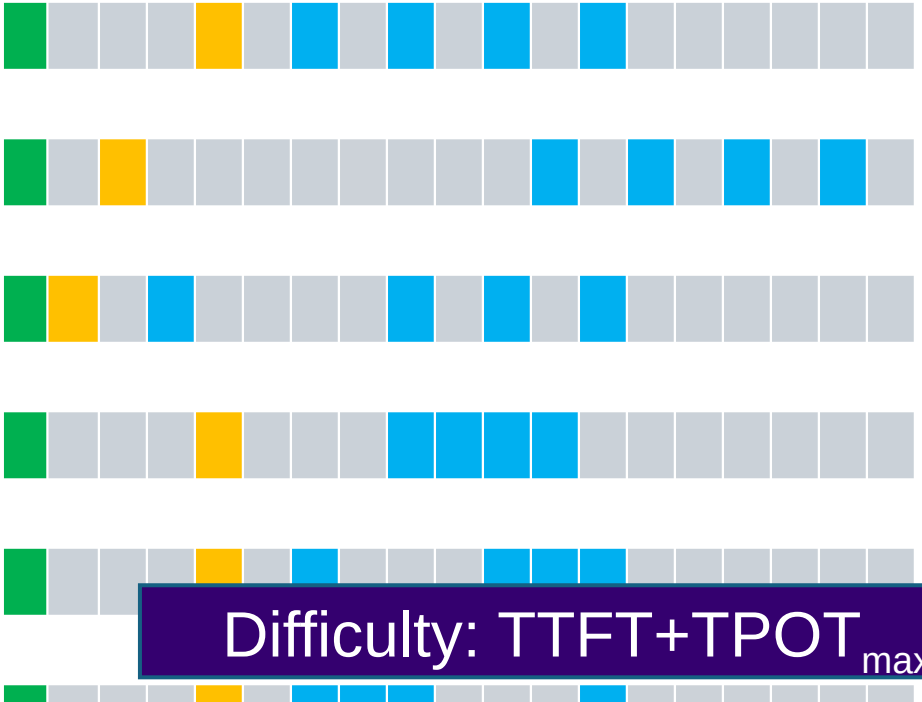
Service Level Agreement

- TTFT + TPOT
- TTFT + Avg TPOT
- End to end time

Service Level Objective

Assumption: TTFT SLO of 6

Arrival Schedule Token

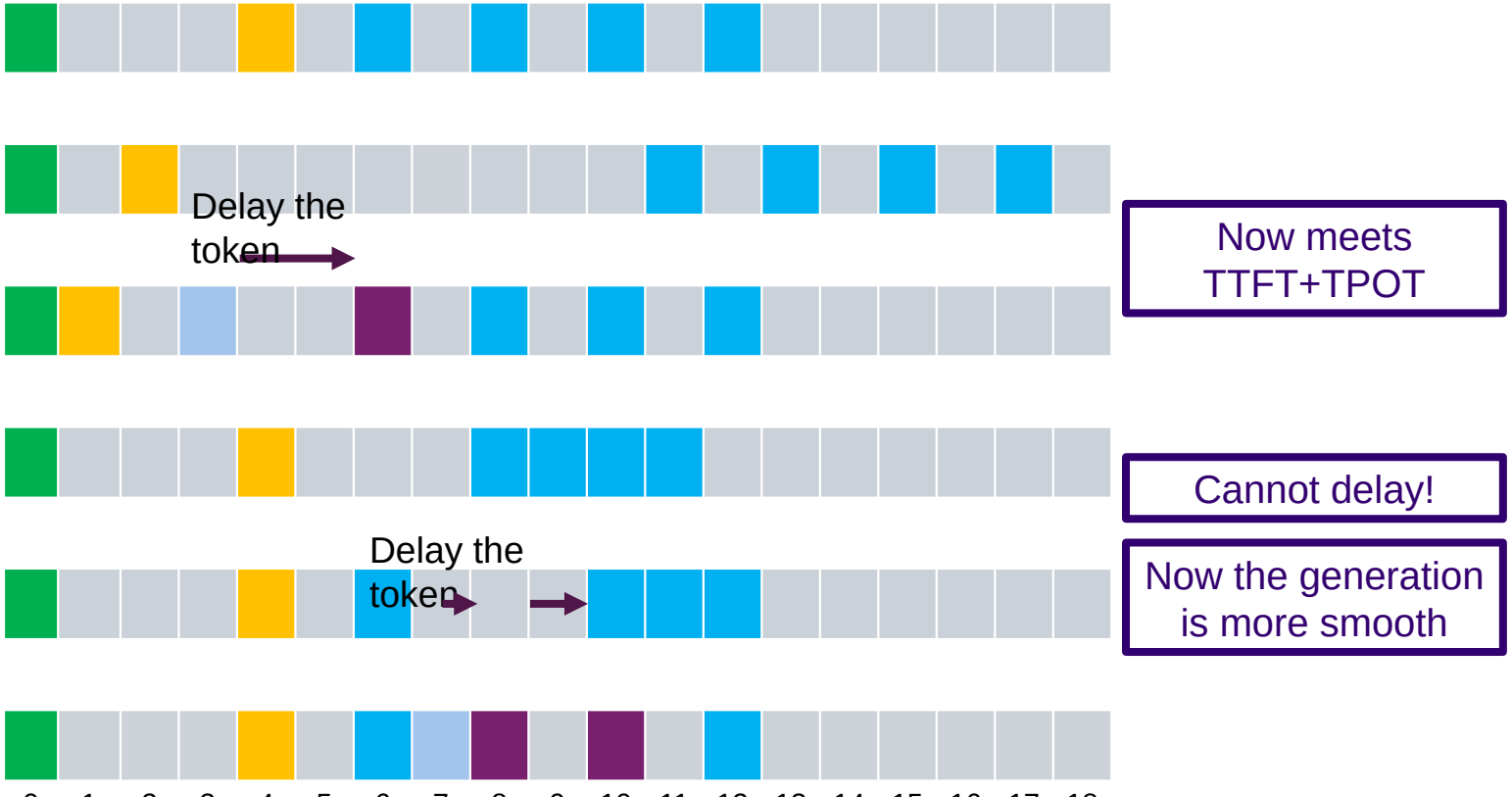


TTFT	$TPOT_{avg}$	$TPOT_{max}$	E2E	TTFT+ $TPOT_{avg}$	TTFT+ $TPOT_{max}$
6	2	2	12		
11	2	2	17	TTFT	TTFT
3	3	5	12	TPOT	TPOT
8	1	1	11	TTFT	TTFT
6	2	4	12		TPOT
6	2	4	12		TPOT

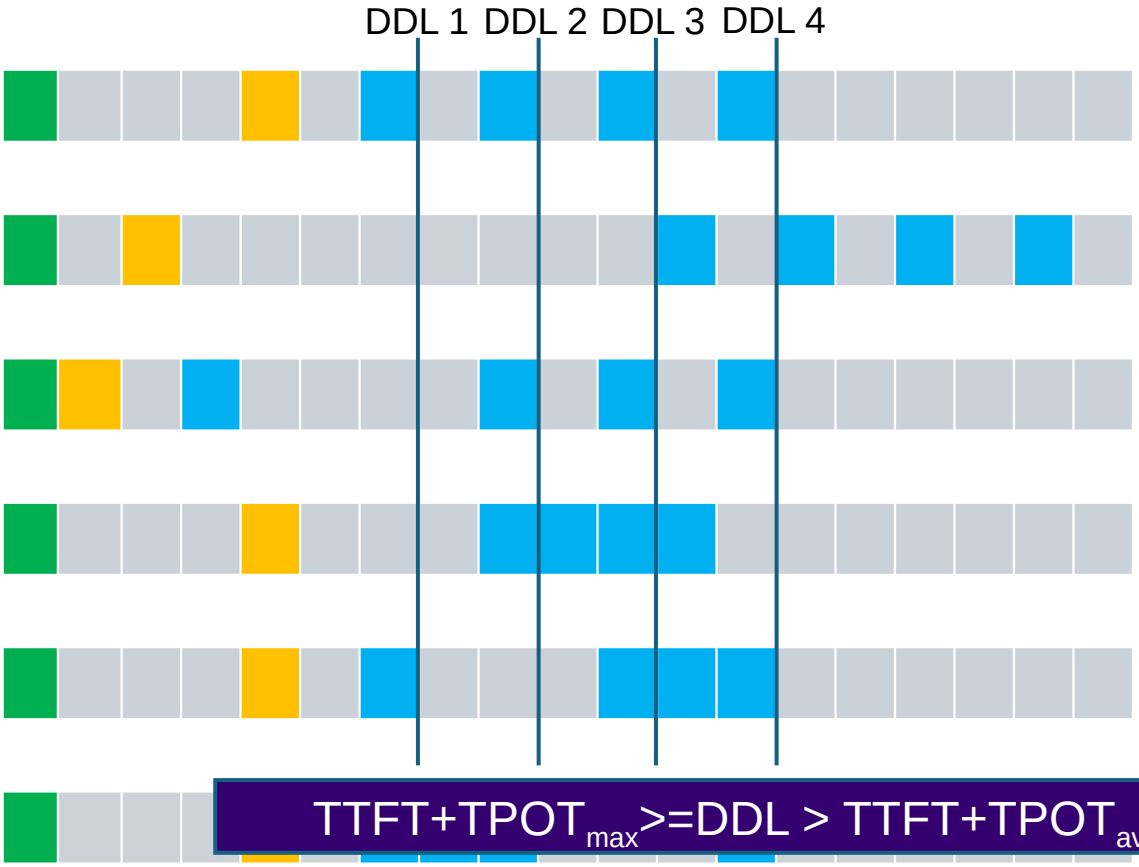
Difficulty: $TTFT+TPOT_{max}>TTFT+TPOT_{avg}>E2E$

Service Level Objective

Delay output and Deadline based SLO



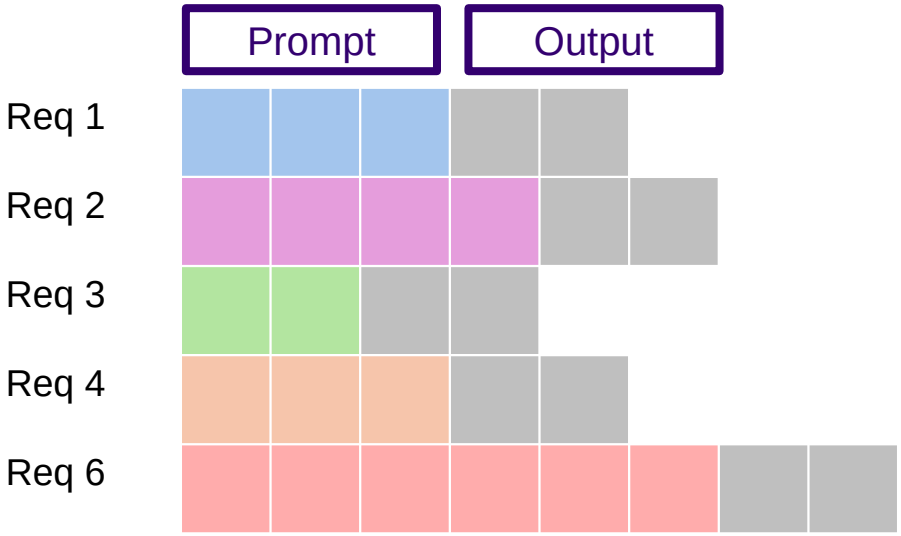
Delay output and Deadline based SLO

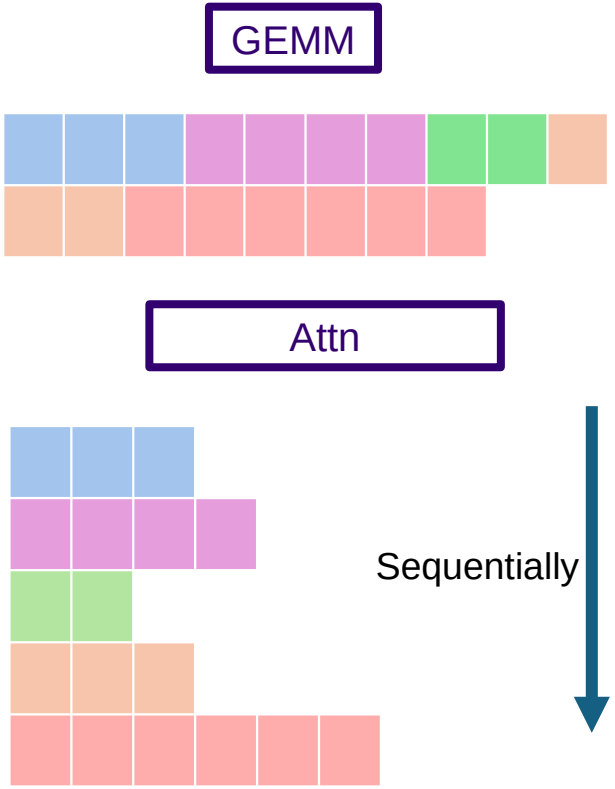


If deadlines are met, we can use delay to achieve $TTFT+TPOT_{max}$

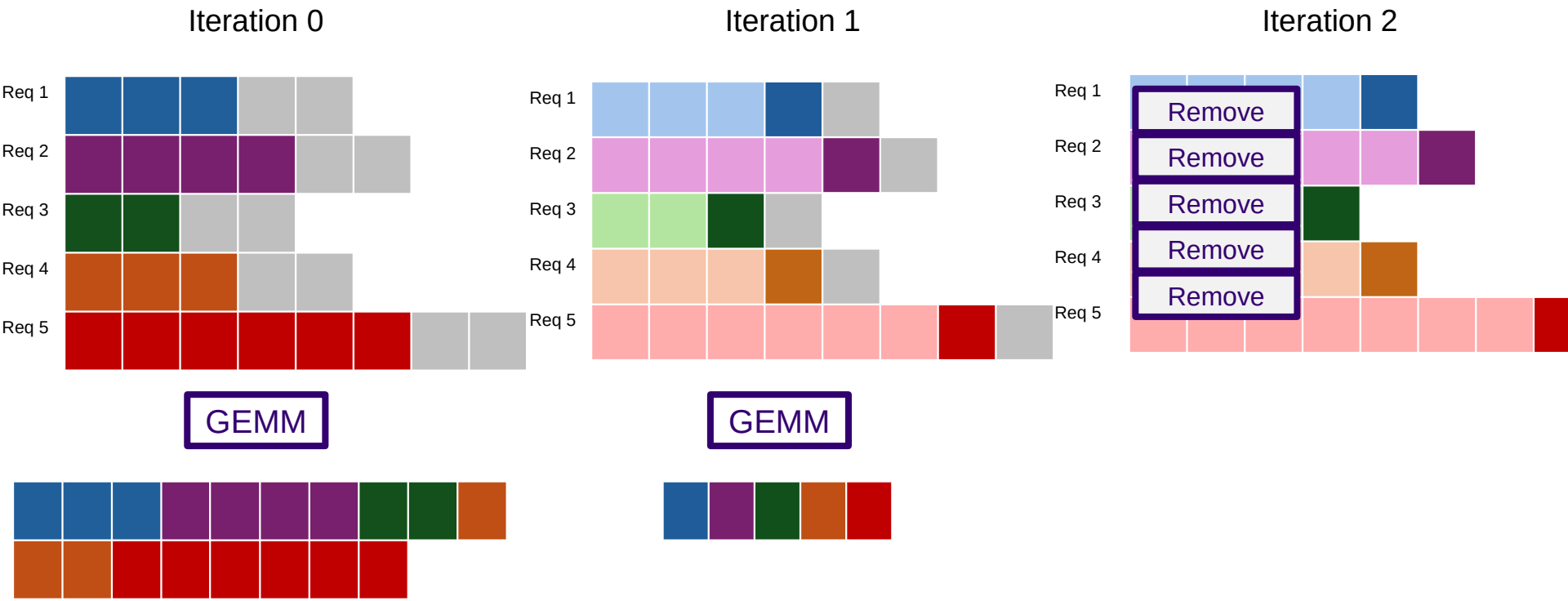
$TTFT+TPOT_{max} >= DDL > TTFT+TPOT_{avg} > E2E$

Batching Requests with Different Input Lengths



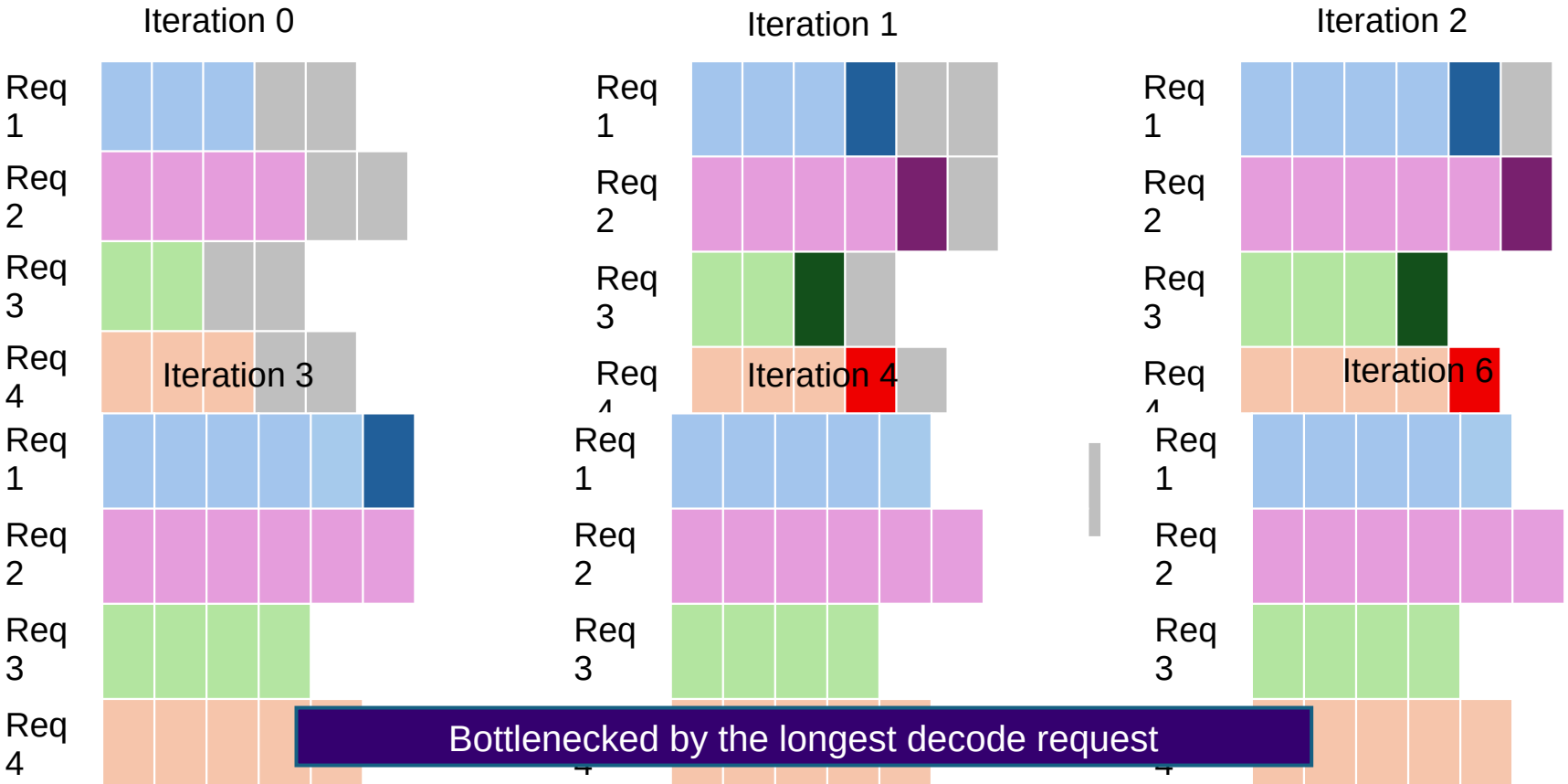


Batching Requests with Different Input Lengths



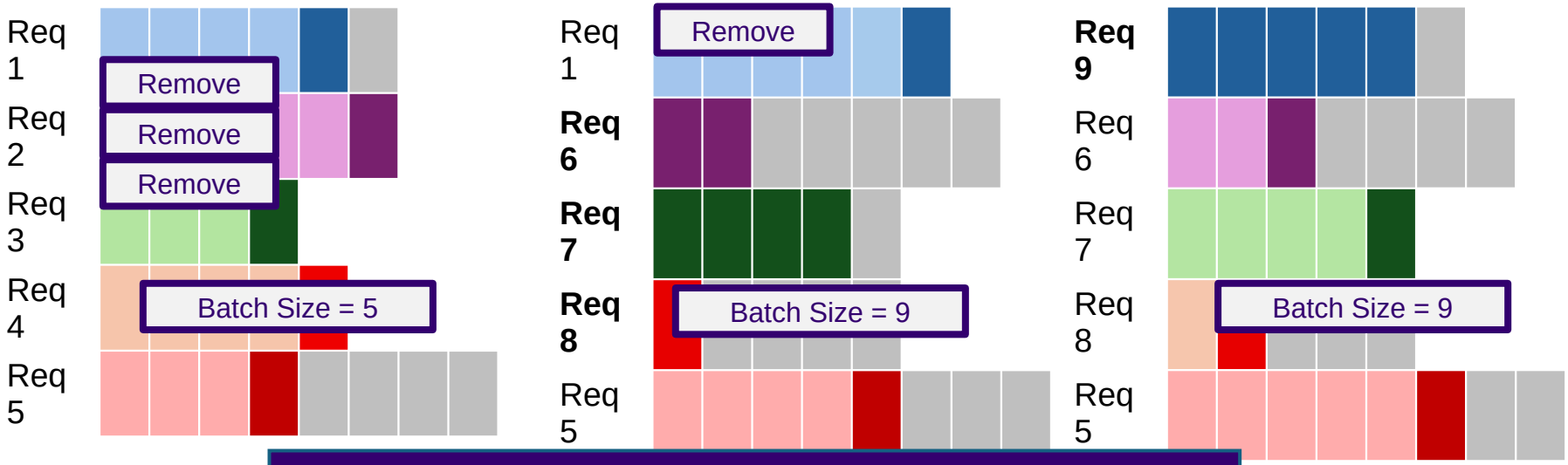
Large batch size for prefill, small batch size for decode

Batching Requests with Different Output Lengths



Orca: Continuous batching

- Key insight: admit new request when the decode requests finish



Now the system achieves a stable batch size and will not be bottlenecked by the longest decoding request

Continuous Batching Batch Size Calculation

- Assume a fixed request batch size B
- Decode length d
- Prefill length p

- A decode request will decode for d iterations
- A prefill request will prefill for 1 cycle

- Within the batch, there are $\frac{1}{d+1}B$ prefill requests, $\frac{d}{d+1}B$ decode requests
- GEMM batch size = $\frac{p}{d+1}B + \frac{d}{d+1}B = \frac{p+d}{d+1}B$

Continuous Batching Batch Size Calculation

- GEMM batch size = $\frac{p}{d+1}B + \frac{d}{d+1}B = \frac{p+d}{d+1}B = B + \frac{p-1}{d+1}B$
- Increasing prefill length will increase GEMM batch size
- Increasing decode length will decrease GEMM batch size

- GEMM batch size $= \frac{p+d}{d+1}B \approx \left(1 + \frac{p}{d}\right)B, d \gg 1$
- Batch size = 512, p/d = 2
  - GEMM batch size = 512*3 = 1536

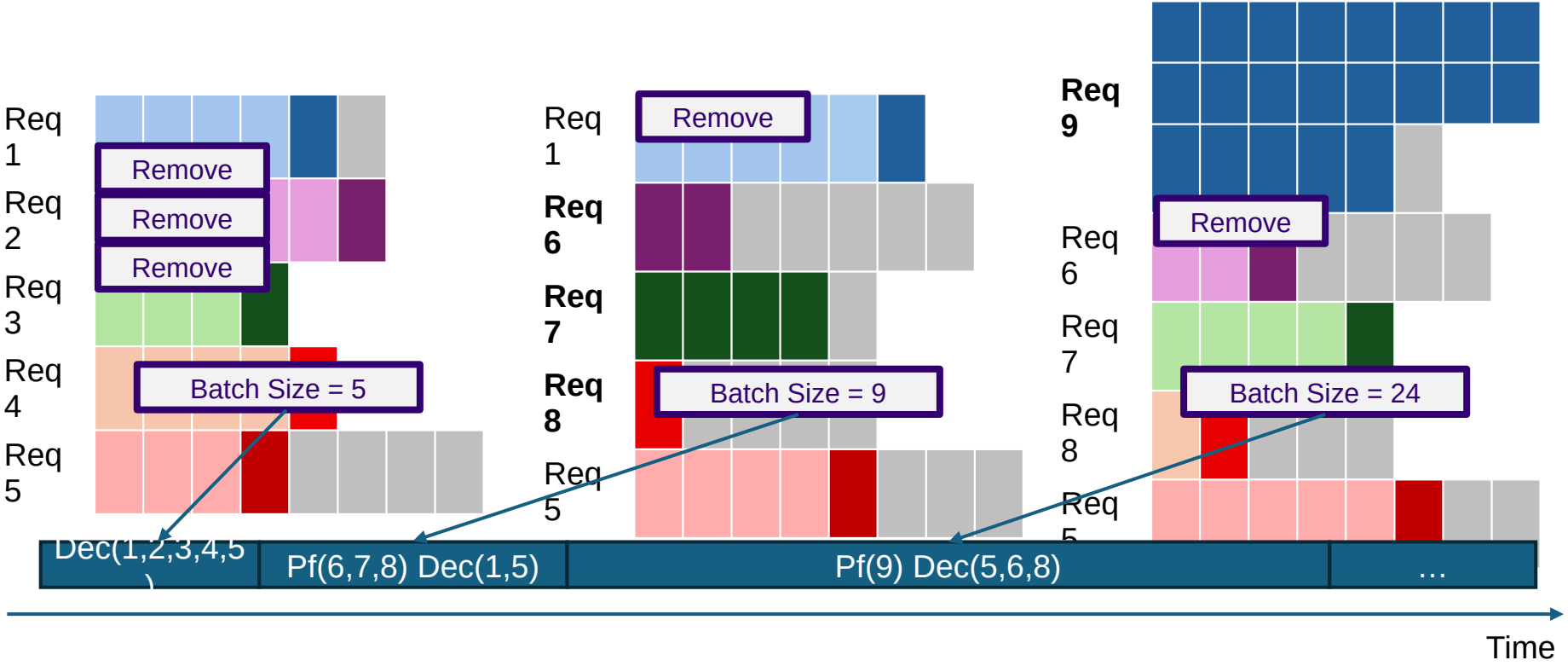
Batch size calculation for various d, p

- Decode length d_i
- Prefill length p_i
- Batch size B
- Request num N
- Total GEMM requirement $\sum (d_i + p_i)$
- Total iteration requirement $\sum(d_i + 1)$
- Each iteration processes a batch of requests, thus after batching, we need $\frac{\sum d_i + 1}{\mathrm{B}}$ iterations
- The GEMM batch size is $\frac{\sum d_i + p_i}{\sum d_i + 1} B = \frac{\mathrm{N}(\bar{d} + \bar{p})}{\mathrm{N}(\bar{d} + 1)} B = \frac{\bar{d} + \bar{p}}{\bar{d} + 1} B$, where $\bar{x}$ means average

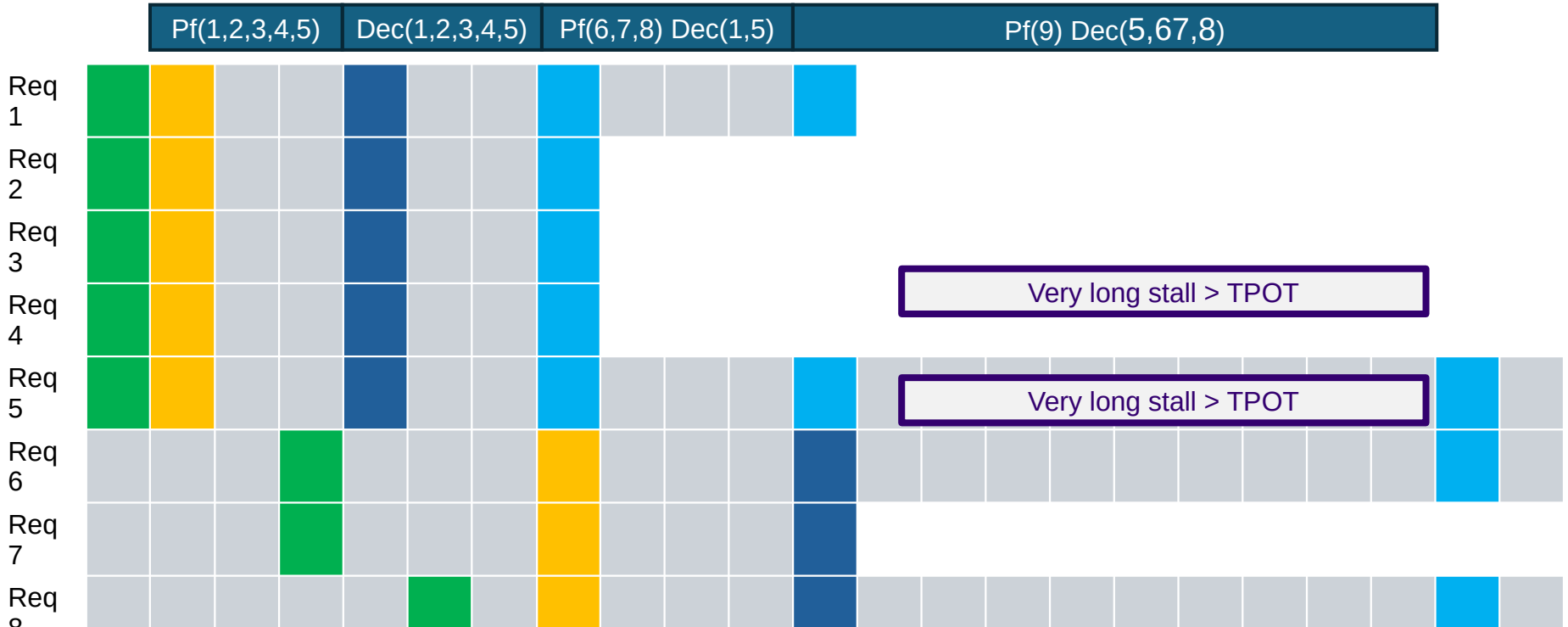
Continuous Batching Performance Analysis

- Higher throughput
  - No need to wait for the longest decode request
  - Stabilize the GEMM batch size
    - GEMM performance is a concave shape
    - GEMM of batch size 1 and 4095 is slower than 2 * GEMM of batch size 2048
- Higher prefill latency
  - Now prefill is batched with decode request, so the prefill speed is reduced
- Higher decode latency
  - Prefill also slows down the concurrent decode
- Reduced queuing time
  - Now, a request can enter the batch at the token granularity

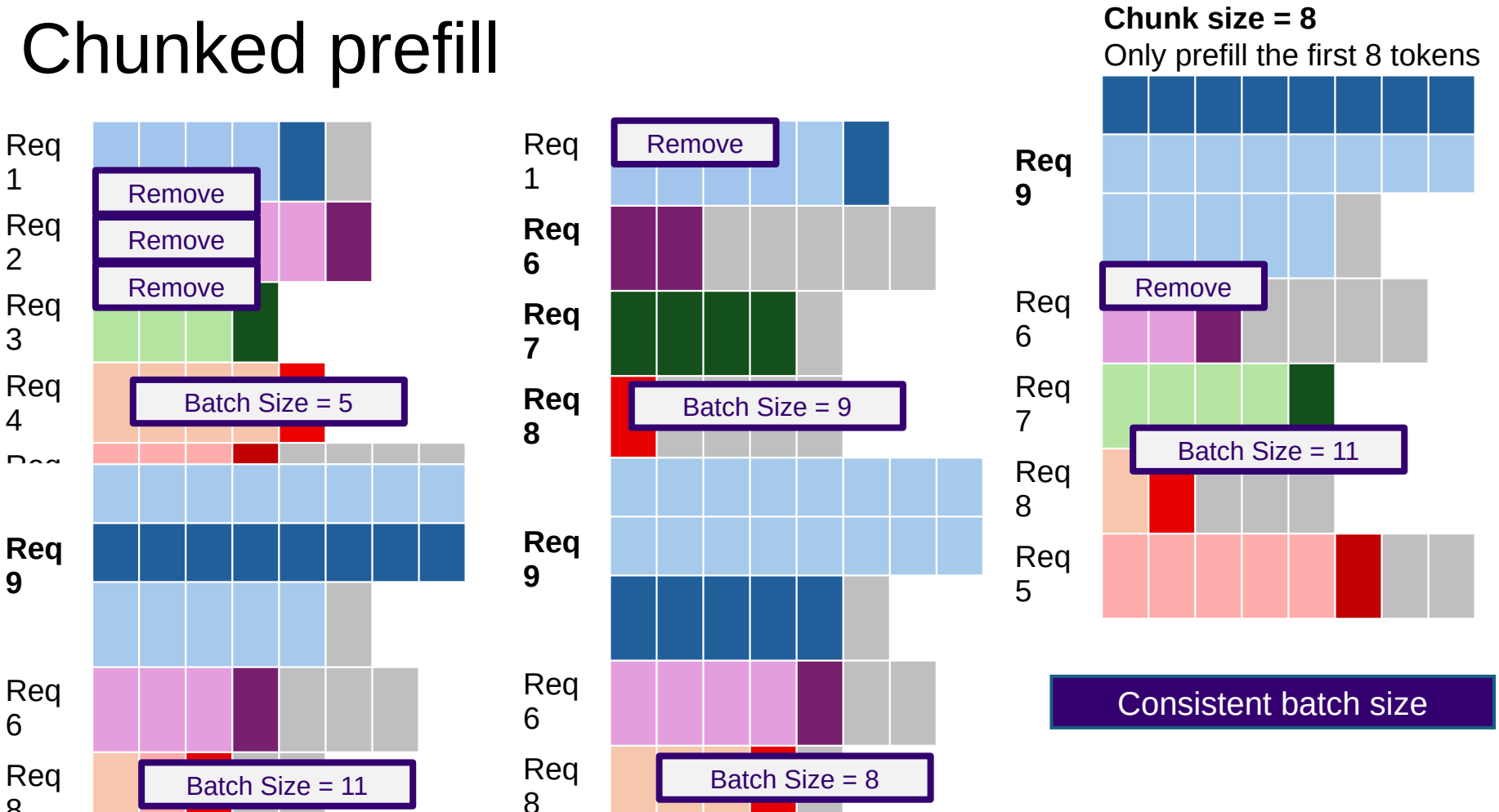
Simple Continuous batching struggles to meet SLO



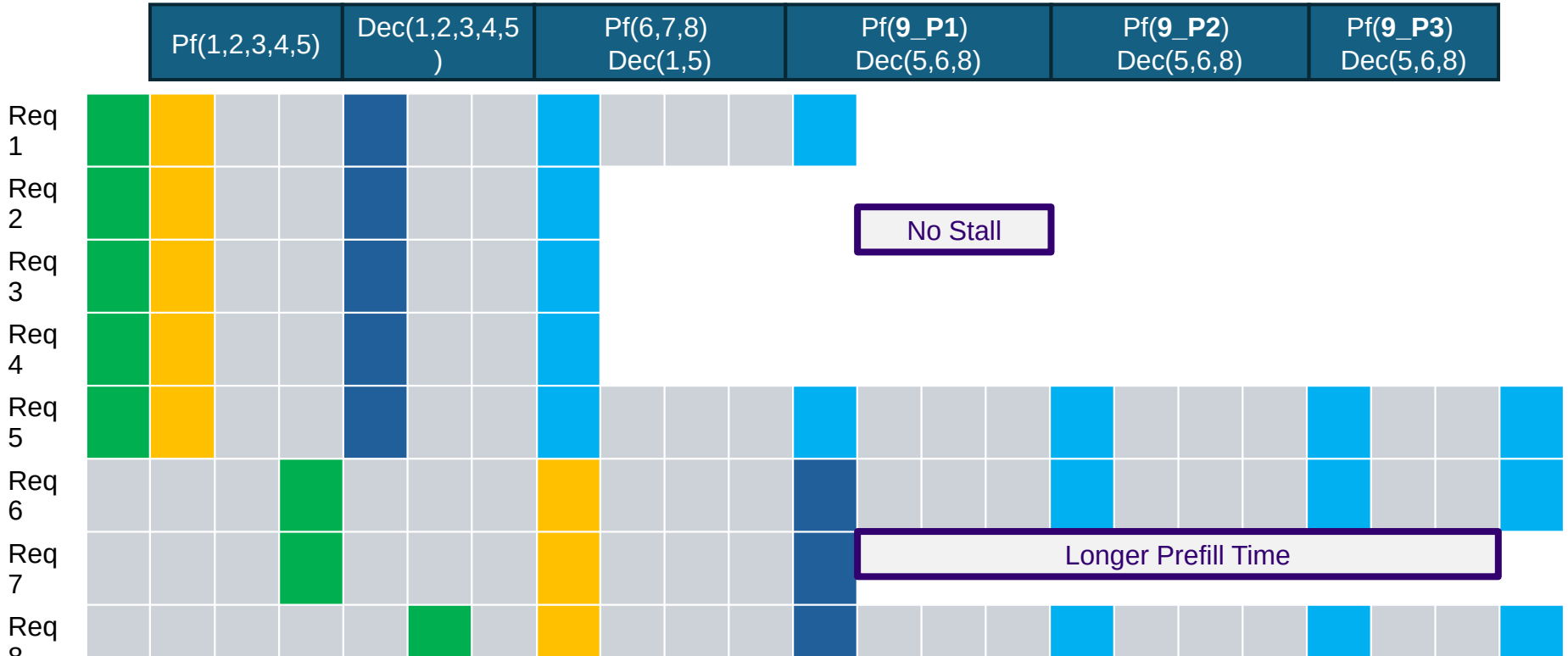
SLO Attainment



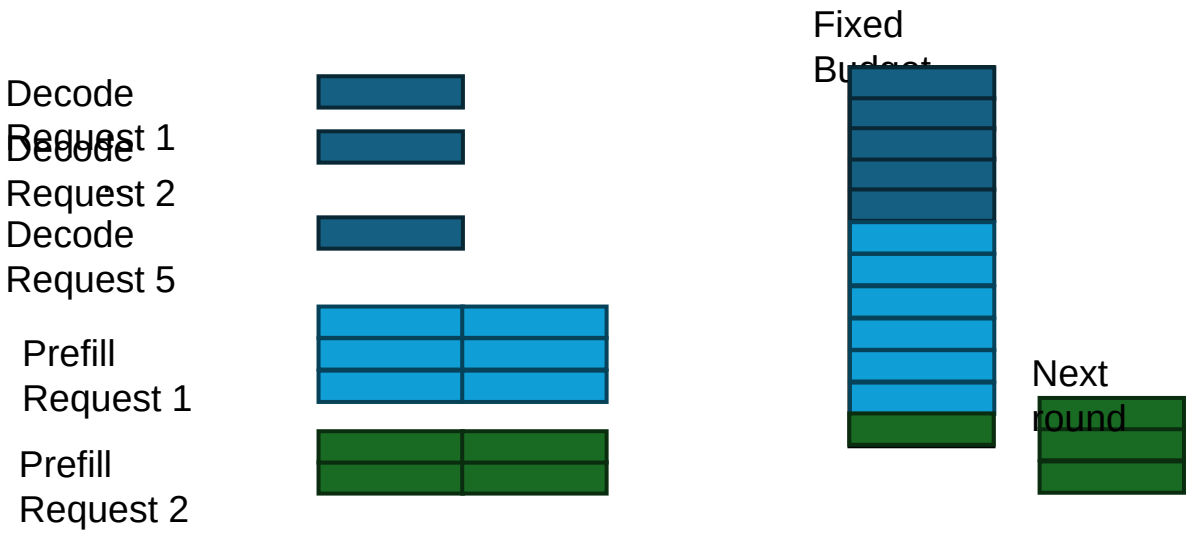
Chunked prefill



Chunked Prefill eliminates generation stall



Fixed Token Budget Chunked Prefill

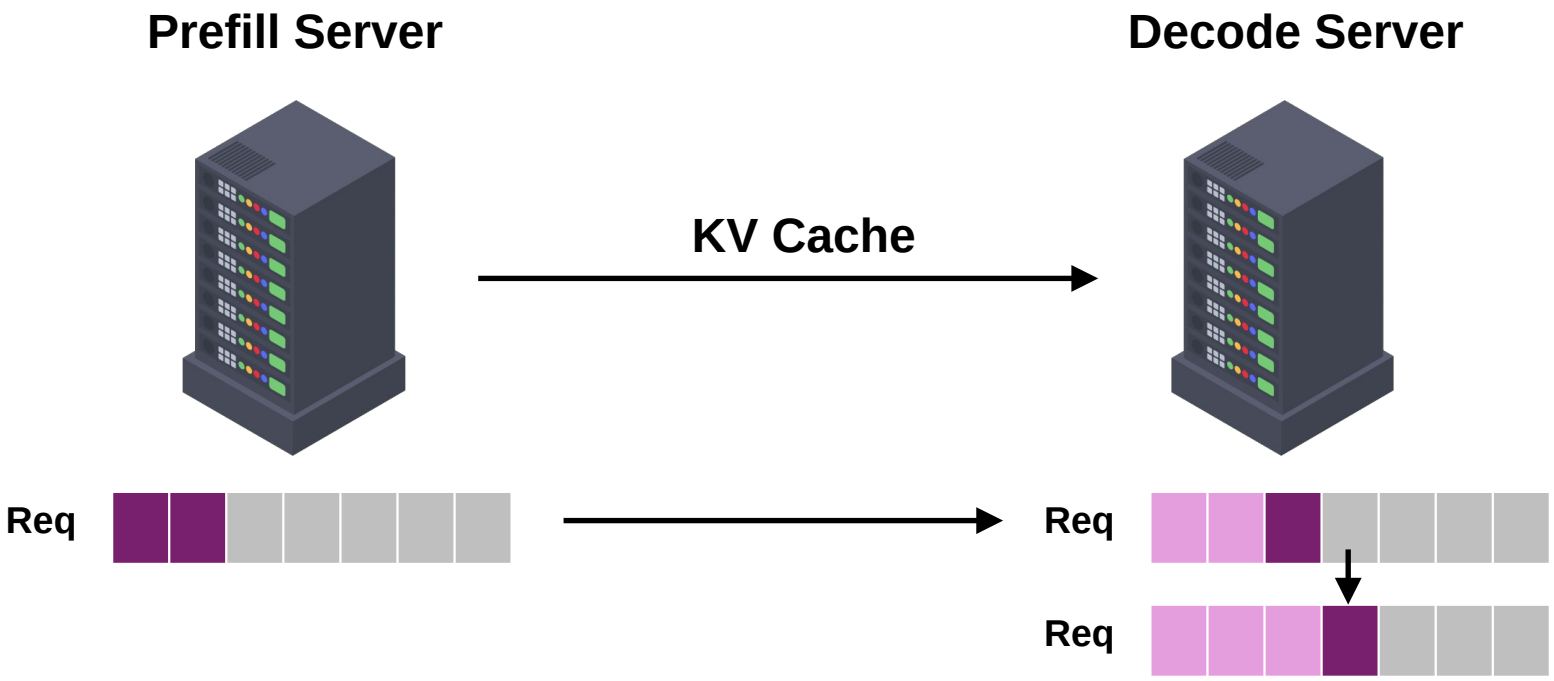


Constant GEMM batch size

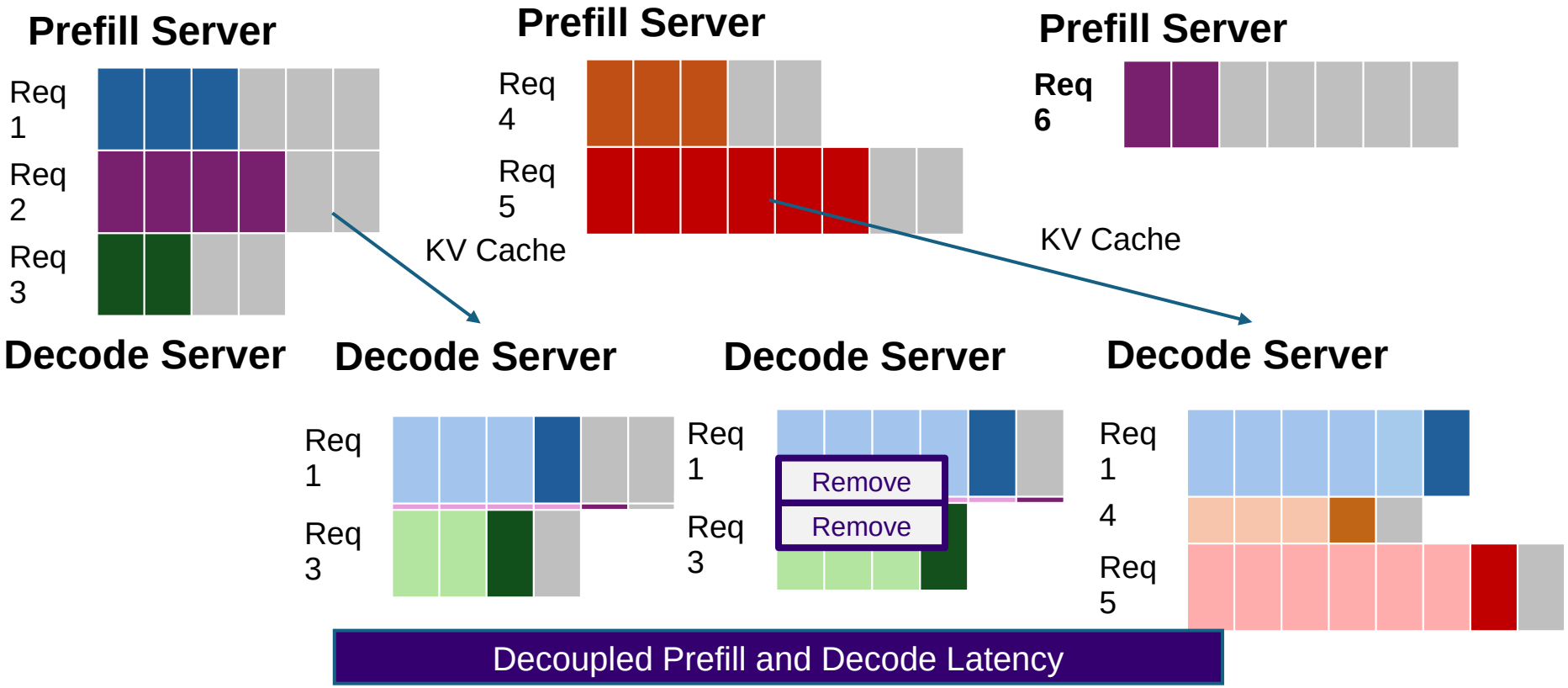
Chunked Prefill Performance Analysis

- Slightly higher throughput
  - Additional KV cache load for chunked prefill
    - When the chunk size is large enough, it is still compute-bound
    - The additional load is negligible
  - Further stabilized the batch size
    - Again, since the TFLOPs curve is concave, GPU utilization is higher
- Further harm the prefill latency
  - Now need to batch with decode requests for more cycles
  - Fractional chunks can add additional latency (e.g., 128 chunk, 257 length)
- Controlled decode latency

Prefill Decode Disaggregation



Prefill Decode Disaggregation



Prefill Decode Disaggregation KV Transfer

- KV cache movement overhead can be significant in PD disaggregation
- PF 16K token
  - 2*2*16K*8*128*32 = 2GB
  - Net = 100Gbps = 12.5GB/s
  - Transfer overhead = 160ms
- However, prefill itself is even slower
  - PF throughput = 16000 token/s
  - Prefill time = 1s
- Therefore, we can use chunked prefill and layer-wise transfer to overlap the KV transfer
- When prefill finishes, only need to transfer the last layer of last chunk

Prefill Decode Disaggregation Performance Analysis

- Throughput is not increased
  - The prefill cluster is fully utilized, while the decode cluster is underutilized
- TTFT and TPOT are decoupled
- By increasing machines, TTFT and TPOT can be reduced to “optimal”, when one request is exclusively executed on one machine

Comparison

Method	Throughput	TTFT	TPOT	Infra Complexity
Simple	Lowest	Short	Short	Low
Continuous Batching	High	Longer	Long, Unstable	Low
Chunked Prefill	Highest	Longest	Long, Controlled	Medium
PD DisAgg	Low	Short	Short	High

Limiting factors of batching

Batching is good for throughput, but there are limits

- SLO
  - Larger batching means higher iteration time, which may violate TTFT and TPOT
- KV cache capacity
  - The available memory for GPU is limited, and the KV cache must fit in GPU memory

Batch Limit due to SLO

- PD disaggregation
  - Prefill batch limit -> TTFT
  - However, the queuing time is also important
    - Requires enough instances to ensure at any given time, a batch can be formed and executed in the near future
  - Decode batch limit -> TPOT
  - B*attn + GEMM(B) + C < TPOT
- Chunked prefill
  - Batch size should consider both TTFT and TPOT
  - Given total B_{dense} , prefill tokens num $\frac{p}{p+d} B_{dense}$, decode token num is $\frac{d}{p+d} B_{dense}$
  - Num cycle for prefill is then $\frac{p+d}{B_{dense}}$
  - $Cycle\ Time$ = GEMM(B_{dense}) + $\frac{d}{p+d} B_{dense} * \text{attn}$
  - Cycle time < TPOT, $\frac{p+d}{B_{dense}} \times Cycle\ Time$ < TTFT

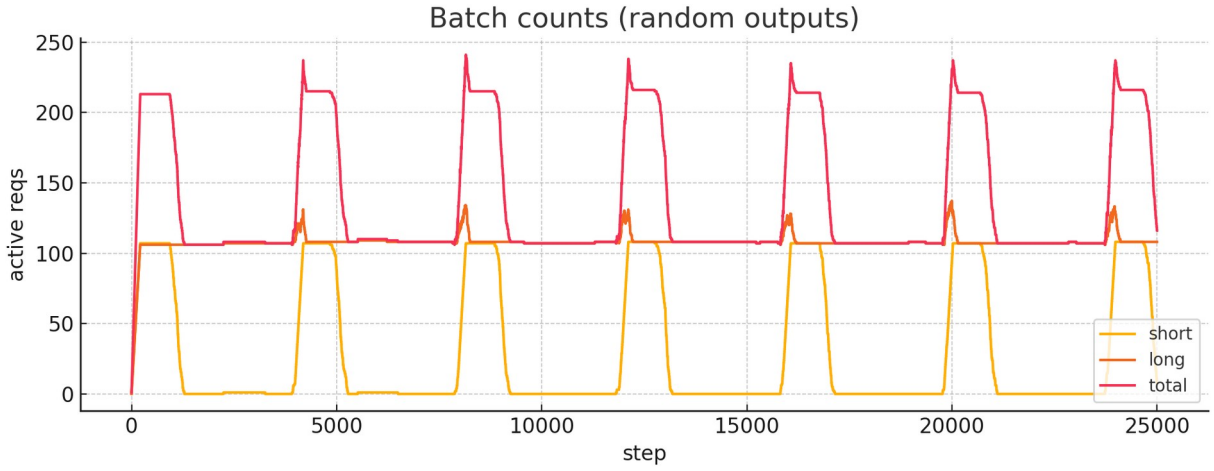
Batch Limit due to GPU capacity

- Total KV cache tokens < limit
- For 8B, H100, num tokens
  - 64*1024*1024*1024/2/2/32/1024 = 524288 = 512K
- Challenges: KV cache can grow over time, and output length is unknown

- Example: 1K input, 1K output
- If scheduled at once, all request reach max length at same time, max batch = 512K / (1K+1K) = 256
- If scheduled uniformly, average length is then 1.5K, max batch = 341

Batch Limit due to GPU capacity

- What if output length is various?
- 1k input, 50% 1k output, 50% 4k output

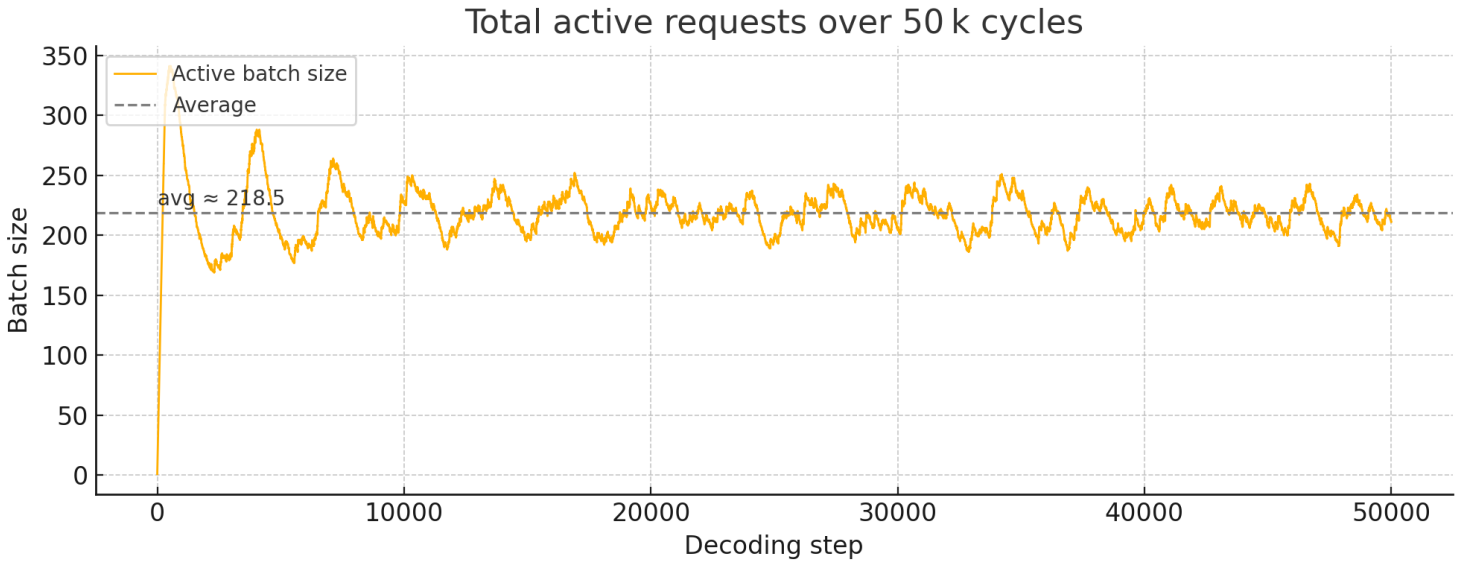


Batch Limit due to GPU capacity

- 1k input, uniform distribution (0-4k) output length
- Average output length = 2k
- Batch size = 512 / (1+2/2) = 256?

Batch Limit due to GPU capacity

- 1k input, uniform distribution (0-4k) output length
- Indeed, batch size is ~ 220
- Longer request stays longer in the KV cache!



Batch size limit for various output length

- KV cache capacity C can deliver in total C*T amount of storage over timeframe T
- A request with prefill length p and decode length d will occupy (p+d/2)*d KV resources
- $\sum(p+d/2)*d = C*T$
- $N((pd)_{avg} + 1/2\ (d^2)_{avg}) = C*T$
- A request will be present in the decode batch for d cycles
- $B \times T = \sum d = N d_{avg}$
- $B = \frac{d_{avg} C}{(pd)_{avg} + \frac{1}{2}(d^2)_{avg}}$

Batch size limit for various output length

- $B = \frac{d_{avg} C}{(pd)_{avg} + \frac{1}{2}(d^2)_{avg}}$
- if constant p, d
  - $B = \frac{C}{p + \frac{1}{2} d}$
- If p, d are independent
  - $B = \frac{C}{p + \frac{1}{2}(d^2)_{avg} / d_{avg}}$
  - 1000 + 16000000/3/2000 = 2333
  - 512k / 2333 = 219

For a continuous uniform random variable $X \sim \mathrm{U}(0, L)$:

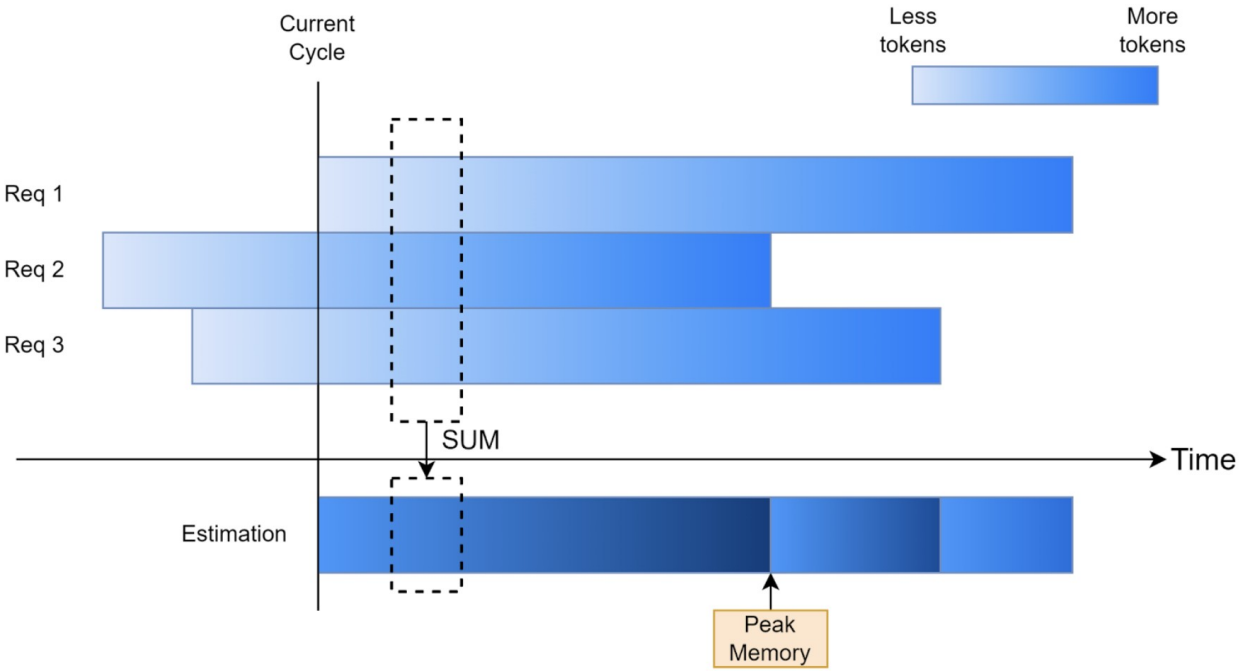
$$\mathbb{E}[X^2] = \frac{1}{L}\int_0^L x^2\,dx = \frac{1}{L}\left[\frac{x^3}{3}\right]_0^L = \frac{L^3}{3L} = \frac{L^2}{3}.$$

Plugging in $L = 4{,}000$ tokens:

$$\mathbb{E}[X^2] = \frac{(4{,}000)^2}{3} = \frac{16{,}000{,}000}{3} \approx 5{,}333{,}333.\overline{3}.$$

So the expected value of X^2 is 5.33×10^6 **tokens²**.

Control the batch size through prediction

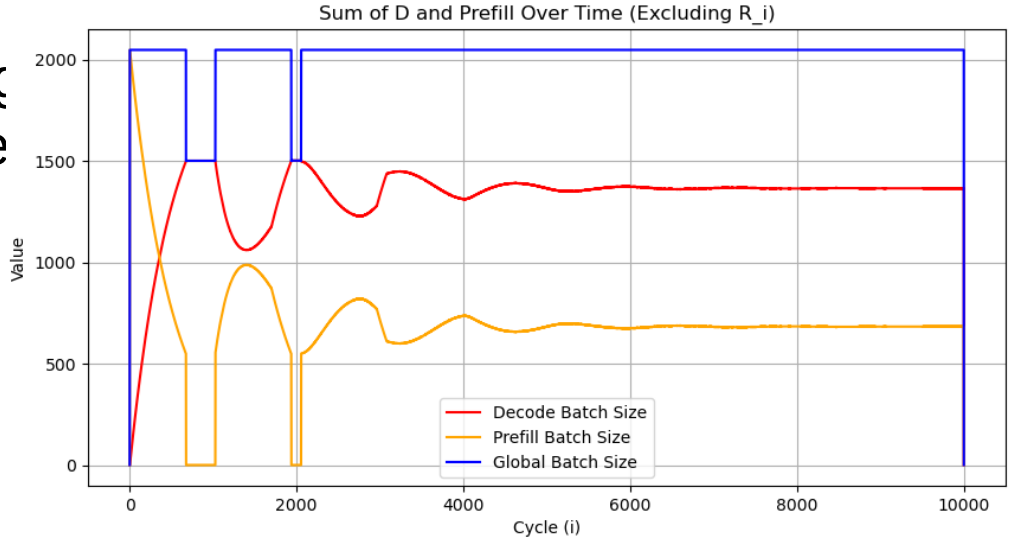


Control batch size for PD Disaggregation

- Add a decode pending queue to hold prefilled requests
- Use prediction to limit the decode batch size

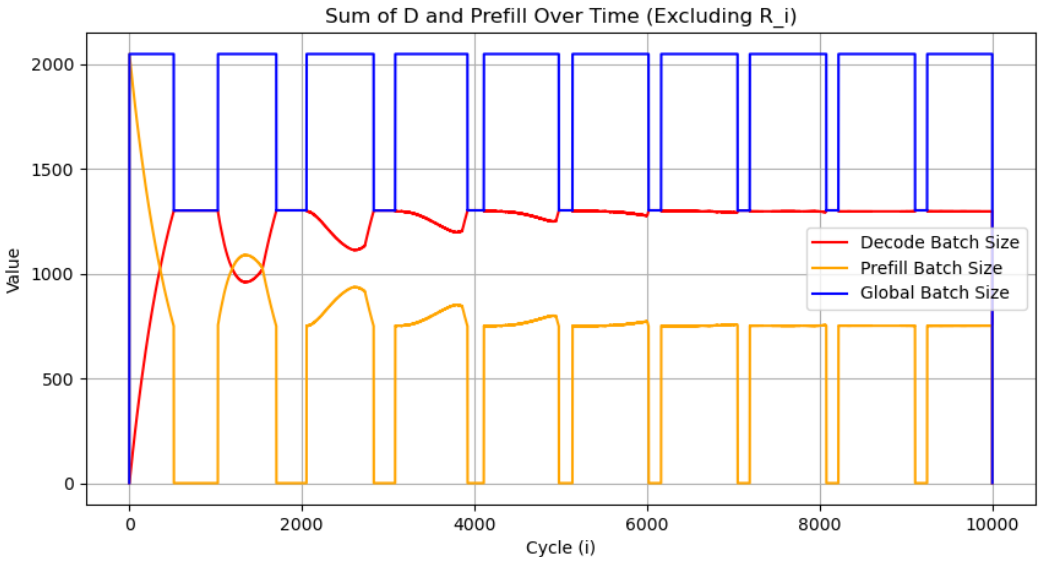
Control batch size for Chunked Prefill

- Stop prefill when the KV cache is predicted to exceed the capacity
- When the KV cache is enoug
  prefill and decode batch size



Control Batch Size for Chunked Prefill

- When
cannot
$\frac{d}{p+d} B_d$
prefill l
oscillat



Output length prediction

- Predicting output length is hard
  - For same input, different trials may have different output length
- Still, rough estimation is possible
  - E.g. translation task will have similar input and output length

- Use a small encoder model to predict length
- Reuse the token embedding of the target model to predict the length

Handling out of memory

- Similar to prefix sharing eviction, when the GPU is actually out of memory during decoding, we need to either evict the KV cache or offload the KV cache
- Again, offloading is faster than recompution

Tricks for “higher” SLO attainment

- To maintain 95% SLO attainment
  - Can we drop the long input/output request?
  - Can we infinitely delay the request if we predict it will not satisfy SLO?
  - Can we only serve higher SLO requests and violate low SLO requests?
- Need additional constraints and SLO evaluation methods
  - Fairness: all request should make progress, and every user get equal share of the throughput
  - SLO violation “Badness”: instead of attain or violate SLO, we calculate the gap between the actual token generation and SLO deadline.